LD0-R3 Density Scene Planning Specification

Bounded Phase-A/Phase-B/Phase-C planning and scene-wide resource accounting

mdstats development specification

2026-07-20

Contents

1 Status and scope

This specification implements architecture gate **LD0-R3** for `mdstats 0.19.42a0`. It is subordinate to the *Dynamical Framework and Atomic Density Architecture Standard*. The gate introduces bounded global planning and resource accounting for all atomic and framework density channels requested by one framework-dynamics scene.

The gate does **not** change:

- cloud-in-cell deposition;
- the `legacy_spectral_v1` Gaussian operator;
- density normalization;
- grid-resolution selection;
- registration or periodic-mean diagnostics;
- highest-density-region thresholds;
- marching-cubes geometry;
- the dense storage backend.

The primary invariant is:

> No floating density field or density mesh may be allocated until every requested density channel has completed exact bounded planning and the scene has passed one global approval decision.

2 Motivation

Before LD0-R3, atomic density fields were allocated before framework density preparation. A later framework resource failure could therefore leave the scene preparation transaction partially executed. Per-channel checks also did not expose a single auditable record of planning bytes, retained field bytes, transient density workspace, mesh bounds, or scene-wide peak memory.

LD0-R3 replaces this sequential allocation policy with

$$\text{Phase A bounds} \longrightarrow \text{Phase B exact indices} \longrightarrow \text{Phase C global approval} \longrightarrow \text{scientific allocation}.$$

3 Scientific and numerical invariants

3.1 Node convention

A logical grid of shape (N_1, N_2, N_3) contains periodic nodes at

$$\mathbf{f}_{ijk} = \left(\frac{i}{N_1}, \frac{j}{N_2}, \frac{k}{N_3} \right).$$

The phase-B occupied-node set is the exact set of logical nodes receiving nonzero cloud-in-cell weight from at least one registered sample, up to floating-point zero weights at samples exactly on a node. The trilinear assignment is the existing Hockney-Eastwood particle-mesh cloud-in-cell rule.

3.2 Dense field storage

For the current dense backend,

$$N_{\text{stored}} = N_{\text{logical}} = N_1 N_2 N_3.$$

The occupied CIC count is diagnostic and planning information; it does not alter the dense allocation.

3.3 Peak-byte accounting

The scene estimate is an explicit package-owned upper bound, not an operating-system RSS prediction. For channel c define

- R_c: retained field bytes after construction;
- T_c: conservative transient workspace bound while constructing that channel;
- P: retained Phase-B planning bytes.

If channels are constructed sequentially in deterministic order, the approved scene peak is

$$B_{\text{peak}} = P + \max_c \left(T_c + \sum_{j<c} R_j \right),$$

and the final retained state is also included:

$$B_{\text{peak}} \leftarrow \max \left(B_{\text{peak}}, P + \sum_c R_c \right).$$

The estimate intentionally excludes arrays owned by the input trajectory and framework topology because they pre-exist this transaction. It includes every new planning array, sample/quadrature array retained during one channel construction, dense scalar array, and conservative FFT workspace owned by density preparation.

4 Data model

4.1 Resource limits

`DensityPlanningLimits` is immutable and schema-versioned. It contains:

```
max_density_fields: int
max_density_voxels: int
max_density_samples: int
max_density_sample_bytes: int
max_density_planning_bytes: int
max_density_stencil_values: int
max_density_nonzero_nodes: int
max_density_stored_block_values: int
max_density_blocks: int
max_density_kernel_pairs: int
max_density_component_values: int
max_density_mesh_cells: int
max_density_mesh_faces: int
max_density_render_points: int
max_density_total_peak_bytes: int
```

`FrameworkDynamicsResources` owns the same public limits and converts them to one `DensityPlanningLimits` record. Existing fields retain their prior defaults.

Sparse-only limits are still validated and serialized, but the dense `legacy_spectral_v1` planner reports zero blocks, zero local kernel pairs, and zero component values.

4.2 Phase-A record

`DensityPhaseAFieldPlan` records conservative metadata-only bounds:

```
field_key: str
source_kind: str
construction_order: int
```

```
sample_count_upper: int
sample_bytes_upper: int
logical_node_count_upper: int
cic_insertions_upper: int
stencil_value_count_upper: int
nonzero_node_count_upper: int
stored_value_count_upper: int
stored_block_count_upper: int
kernel_pair_count_upper: int
component_value_count_upper: int
mesh_cell_count_upper: int
mesh_face_count_upper: int
retained_bytes_upper: int
transient_bytes_upper: int
```

Phase A may use exact counts when they are available from metadata, but every value is semantically an upper bound.

4.3 Phase-B record

`DensityPhaseBFieldPlan` records exact dense-backend planning values:

```
field_key: str
source_kind: str
construction_order: int
sample_count: int
sample_bytes: int
grid_shape: tuple[int, int, int]
logical_node_count: int
occupied_cic_node_indices: ndarray[int64]
nonzero_node_count_upper: int
stored_value_count: int
mesh_cell_count: int
mesh_face_count_upper: int
planning_bytes: int
retained_bytes: int
transient_bytes_upper: int
```

`occupied_cic_node_indices` is sorted, unique, C-contiguous, read-only, and flattened with `numpy.ravel_multi_index(..., order="C")`.

The phase-B record contains no floating density values and no mesh arrays.

4.4 Scene plan

`DensityScenePlan` contains:

```
phase_a_fields: tuple[DensityPhaseAFieldPlan, ...]
phase_b_fields: tuple[DensityPhaseBFieldPlan, ...]
limits: DensityPlanningLimits
phase_a_approved: bool
phase_b_approved: bool
phase_c_approved: bool
planning_bytes: int
retained_bytes: int
estimated_peak_bytes: int
metadata: FrozenJSONMapping
```

The record is attached to `FrameworkDynamicsScene.planning_record` and summarized in scene metadata.

5 Planning algorithms

5.1 Phase A

Phase A runs before allocating registered sample arrays or integer node-index arrays. For each requested field it computes:

1. exact or conservative source-sample count;
2. sample byte bound;
3. logical-node upper bound from the field's available dense voxel budget;
4. at most eight CIC insertion indices per sample;
5. dense stored-value bound;
6. periodic logical-cell count;
7. at most five marching-cubes triangles per cell per configured shell;
8. conservative retained and transient byte bounds.

For framework edge quadrature, Phase A may use `max_density_samples` as the bound because the exact count depends on Cartesian segment lengths. Phase B computes the exact count before any floating field allocation.

Every Phase-A hard-limit failure raises `GraphComplexityError` immediately.

5.2 Phase B atomic fields

For each atomic selection:

1. resolve its atom indices and deterministic field label;
2. construct registered display fractional coordinates;
3. resolve the existing dense numerics under the same per-field voxel budget used by `0.19.41a0`;
4. fold coordinates into $[0, 1)^3$;
5. construct and deduplicate the eight CIC target-node indices;
6. record exact sample, node, storage, mesh, and byte counts.

The phase-B registered coordinate array is temporary and is released after its integer node plan is constructed. Only the integer occupied-node array is retained in the scene plan.

5.3 Phase B framework fields

Framework vertex planning uses the already registered projected-vertex coordinates. Framework edge planning streams the existing midpoint quadrature rule:

$$n_e = \max\left(1, \left\lceil \frac{L_e}{h_q} \right\rceil\right),$$

constructs one segment's midpoint coordinates at a time, and accumulates occupied CIC nodes without retaining the complete edge sample cloud.

The exact quadrature count, total sample bytes, and occupied nodes are recorded.

5.4 Phase C approval

All phase-B records are collected before any field constructor is called. The scene then verifies:

- field count;
- total dense stored values and compatibility voxel limit;
- per-field and scene sample limits;
- sample bytes;
- planning bytes;
- nonzero-node upper bounds;
- stencil, block, kernel-pair, component, mesh-cell, mesh-face, and render-point limits;
- estimated scene peak bytes.

Only one fully approved `DensityScenePlan` permits scientific allocation.

6 Dense workspace model

For `legacy_spectral_v1`, the conservative transient workspace is

$$T_c = 256N_c + 64S_c + B_{\text{samples},c},$$

where N_c is the logical-node count and S_c is the exact sample count. The coefficient covers the dense mass grid, FFT complex arrays, Cartesian reciprocal coordinates, kernel, squared wave numbers, products, inverse transform, and density normalization intermediates with margin.

Retained bytes are

$$R_c = 8N_c + B_{\text{stored sample positions},c}.$$

This formula is intentionally conservative and is tested against explicit package allocation accounting on representative atomic, framework-vertex, and framework-edge fixtures.

7 Execution and realization checks

After Phase C approval, existing dense preparation executes in the phase-B construction order. Every realized field is checked against its approved plan:

- field key;
- logical shape;
- stored values;
- nonzero values not exceeding the approved upper bound;
- retained bytes not exceeding the approved retained bound;
- sample count metadata when available.

An underestimate raises `GraphComplexityError` and is treated as a planner defect.

8 API and compatibility

The following additions are public:

```
DensityPlanningLimits
DensityPhaseAFieldPlan
DensityPhaseBFieldPlan
DensityScenePlan
plan_density_scene
```

Existing direct calls to `prepare_atomic_density_fields()` and `prepare_framework_density_fields()` remain supported with their legacy local resource checks. Global transactionality is guaranteed by `prepare_framework_dynamics_scene`.

The default dense numerical results must remain exactly equal to `0.19.41a0` for matched fixtures.

9 Failure semantics

All planning-limit failures raise `GraphComplexityError` and identify:

- the phase;
- the field or scene;
- the estimated/exact value;
- the violated limit name and value.

No planner failure changes grid resolution, Gaussian bandwidth, selections, quadrature spacing, shell fractions, or backend.

10 Focused tests

10.1 Contract tests

- schema validation and JSON round trips;
- read-only sorted occupied-node arrays;
- deterministic field order and byte accounting;
- scene plan immutability.

10.2 Limit tests

Every hard limit is exercised by a pre-allocation failure test. Sparse-only limits are tested against explicit nonzero synthetic plan records until their owning backend is implemented.

10.3 Transaction tests

- monkeypatch the dense field constructor and prove it is never called when any Phase-A or Phase-B channel fails;
- request atomic plus framework channels and prove all phase-B plans exist before the first constructor call;
- verify deterministic approval identifiers and metadata.

10.4 Accounting tests

- phase-A bounds are never below phase-B exact counts;
- phase-B approved counts are never below realized field counts;
- approved peak bytes are not below package-owned realized allocation accounting;
- overestimation is recorded.

10.5 Numerical compatibility

Matched `0.19.41a0` and `0.19.42a0` fixtures must have exactly equal:

- atomic density arrays;
- framework-vertex arrays;
- framework-edge arrays;
- integrals;
- 50%, 80%, and 95% HDR thresholds;
- marching-cubes vertices and faces.

11 Acceptance gate

LD0-R3 passes only if:

1. every hard limit has a focused pre-allocation failure test;
2. no floating field or mesh allocation occurs before all requested channels pass Phase B;
3. exact index planning is bounded by `max_density_planning_bytes`;
4. Phase-A values are never below Phase-B values;
5. Phase-B approved hard counts are never below realized counts;
6. estimated package-owned peak bytes are at least realized package-owned peak bytes on the required benchmarks;
7. default dense scientific values and meshes remain exactly compatible with `0.19.41a0`.

12 References

1. Hockney, R. W., and J. W. Eastwood. *Computer Simulation Using Particles*. Taylor & Francis, 1988. The existing cloud-in-cell assignment is retained; LD0-R3 adds planning around it rather than changing the assignment algorithm.